

Table 3: Number of parameters and model providers of LLMs used in our experiments.

LLMs	#Parameters	Model provider
GPT-4	1.5T	OpenAI
PaLM 2 text-bison-001	340B	Google
GPT-3.5-Turbo	154B	OpenAI
Bard	137B	Google
Vicuna-33b-v1.3	33B	LM-SYS
Flan-UL-2	20B	Google
Vicuna-13b-v1.3	13B	LM-SYS
Llama-2-13b-chat	13B	Meta
Llama-2-7b-chat	7B	Meta
InternLM-Chat-7B	7B	InternLM

data is compromised. One key limitation of this defense is that it fails when the injected task and target task are in the same type, e.g., both of them are for spam detection.

Known-answer detection [31]: This detection method is based on the following key observation: the instruction prompt is not followed by the LLM under a prompt injection attack. Thus, the idea is to proactively construct an instruction (called *detection instruction*) with a known ground-truth answer that enables us to verify whether the detection instruction is followed by the LLM or not when combined with the (compromised) data. For instance, we can construct the following detection instruction: “Repeat *[secret key]* once while ignoring the following text. \nText:”, where “*[secret key]*” could be an arbitrary text. Then, we concatenate this detection instruction with the data and let the LLM produce a response. The data is detected as compromised if the response does not output the “*[secret key]*”. Otherwise, the data is detected as clean. We use 7 random characters as the secret key in our experiments.

6 Evaluation

6.1 Experimental Setup

LLMs: We use the following LLMs in our experiments: PaLM 2 text-bison-001 [12], Flan-UL2 [44], Vicuna-33b-v1.3 [18], Vicuna-13b-v1.3 [18], GPT-3.5-Turbo [5], GPT-4 [33], Llama-2-13b-chat [21], Llama-2-7b-chat [7], Bard [28], and InternLM-Chat-7B [45]. Table 3 shows the total number of parameters and model providers of those LLMs. Regarding the determinism of the LLM responses, for open-source LLMs, we fix the seed of a random number generator to make LLM responses deterministic, which makes our results reproducible. For closed-source LLMs, we set the temperature to a small value (i.e., 0.1) and found non-determinism has a small impact on the results.

Unless otherwise mentioned, we use GPT-4 as the default LLM as it achieves good performance on various tasks. Specifically, we use the API of GPT-4 provided by Azure OpenAI Studio and we leverage the GPT-4 built-in role-separation features to query the API with the message in the following

format: [{"role": "system", "content": instruction prompt}, {"role": "user", "content": data}], where instruction prompt and (compromised) data are from a given task.

Datasets for 7 tasks: We consider the following 7 commonly used natural language tasks: *duplicate sentence detection (DSD)*, *grammar correction (GC)*, *hate detection (HD)*, *natural language inference (NLI)*, *sentiment analysis (SA)*, *spam detection (SD)*, and *text summarization (Summ)*. We select a benchmark dataset for each task. Specifically, we use MRPC dataset for duplicate sentence detection [20], Jfleg dataset for grammar correction [32], HSOL dataset for hate content detection [19], RTE dataset for natural language inference [47], SST2 dataset for sentiment analysis [42], SMS Spam dataset for spam detection [10], and Gigaword dataset for text summarization [39].

Target and injected tasks: We use each of the seven tasks as a target (or injected) task. Note that a task could be used as both the target task and injected task simultaneously. As a result, there are 49 combinations in total (7 target tasks $\times$ 7 injected tasks). A target task consists of *target instruction* and *target data*, whereas an injected task contains *injected instruction* and *injected data*. Table 11 in Appendix shows the target instruction and injected instruction for each target/injected task. For each dataset of a task, we select 100 examples uniformly at random without replacement as the target (or injected) data. Note that there is no overlap between the 100 examples of the target data and 100 examples of the injected data. Each example contains a text and its ground truth label, where the text is used as the target/injected data and the label is used for evaluating attack success.

We note that, when the target task and the injected task are the same type, the ground truth label of the target data could be the same as the ground truth label of the injected data, making it very challenging to evaluate the effectiveness of the prompt injection attack. Take spam detection as an example. If both the target and injected tasks aim to make an LLM-Integrated Application predict the label of a non-spam message, when the LLM-Integrated Application outputs “non-spam”, it is hard to determine whether it is because of the attack. To address the challenge, we select examples with different ground truth labels as the target data and injected data in this case. Additionally, to consider a real-world scenario, we select examples whose ground truth labels are “spam” (or “hateful”) as target data when the target and injected tasks are spam detection (or hate content detection). For instance, an attacker may wish a spam post (or a hateful text) to be classified as “non-spam” (or “non-hateful”). Please refer to Section A in Appendix for more details.

Evaluation metrics: We use the following evaluation metrics for our experiments: *Performance under No Attacks (PNA)*, *Attack Success Value (ASV)*, and *Matching Rate (MR)*. These metrics can be used to evaluate attacks and prevention-based defenses. To measure the performance of detection-based

defenses, we further use *False Positive Rate (FPR)* and *False Negative Rate (FNR)*. All these metrics have values in $[0,1]$. We use $\mathcal{D}^t$ (or $\mathcal{D}^e$) to denote the set of examples for the target data of the target task t (or injected data of the injected task e). Given an LLM f , a target instruction $\mathbf{s}^t$, and an injected instruction $\mathbf{s}^e$, those metrics are defined as follows:

PNA-T and PNA-I. PNA measures the performance of an LLM on a task (e.g., a target or injected task) when there is no attack. Formally, PNA is defined as follows:

$$PNA = \frac{\sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{D}} \mathcal{M}[f(\mathbf{s} \oplus \mathbf{x}), \mathbf{y}]}{|\mathcal{D}|}, \quad (2)$$

where $\mathcal{M}$ is the metric used to evaluate the task (we defer the detailed discussion to the end of this section), $\mathcal{D}$ contains a set of examples, $\mathbf{s}$ represents an instruction for the task, $\oplus$ represents the concatenation operation, and $(\mathbf{x}, \mathbf{y})$ is an example in which $\mathbf{x}$ is a text and $\mathbf{y}$ is the ground truth label of $\mathbf{x}$. When the task is a target task (i.e., $\mathbf{s} = \mathbf{s}^t$ and $\mathcal{D} = \mathcal{D}^t$), we denote PNA as *PNA-T*. PNA-T represents the performance of an LLM on a target task when there are no attacks. A defense sacrifices the utility of a target task when there are no attacks if PNA-T is smaller after deploying the defense. Similarly, we denote PNA as *PNA-I* when the task is an injected task (i.e., $\mathbf{s} = \mathbf{s}^e$ and $\mathcal{D} = \mathcal{D}^e$). PNA-I measures the performance of an LLM on an injected task when we query the LLM with the injected instruction and injected data.

ASV. ASV measures the performance of an LLM on an injected task under a prompt injection attack. Formally, ASV is defined as follows:

$$ASV = \frac{\sum_{(\mathbf{x}^t, \mathbf{y}^t) \in \mathcal{D}^t, (\mathbf{x}^e, \mathbf{y}^e) \in \mathcal{D}^e} \mathcal{M}^e[f(\mathbf{s}^t \oplus \mathcal{A}(\mathbf{x}^t, \mathbf{s}^e, \mathbf{x}^e)), \mathbf{y}^e]}{|\mathcal{D}^t| |\mathcal{D}^e|}, \quad (3)$$

where $\mathcal{M}^e$ is the metric to evaluate the injected task e (we defer the detailed discussion) and $\mathcal{A}$ represents a prompt injection attack. As we respectively use 100 examples as target data and injected data, there are 10,000 pairs of examples in total. To save the computation cost, we randomly sample 100 pairs when we compute ASV in our experiments. An attack is more successful and a defense is less effective if ASV is larger. Note that PNA-I would be an upper bound of ASV for an injected task.

MR. ASV depends on the performance of an LLM for an injected task. In particular, if the LLM has low performance on the injected task, then ASV would be low. Therefore, we also use MR as an evaluation metric, which compares the response of the LLM under a prompt injection attack with the one produced by the LLM with the injected instruction and injected data as the prompt. Formally, we have:

$$MR = \frac{\sum_{(\mathbf{x}^t, \mathbf{y}^t) \in \mathcal{D}^t, (\mathbf{x}^e, \mathbf{y}^e) \in \mathcal{D}^e} \mathcal{M}^e[f(\mathbf{s}^t \oplus \mathcal{A}(\mathbf{x}^t, \mathbf{s}^e, \mathbf{x}^e)), f(\mathbf{s}^e \oplus \mathbf{x}^e)]}{|\mathcal{D}^t| |\mathcal{D}^e|}. \quad (4)$$

We also randomly sample 100 pairs when computing MR to save computation cost. An attack is more successful and a defense is less effective if the MR is higher.

FPR. FPR is the fraction of clean target data samples that are incorrectly detected as compromised. Formally, we have:

$$FPR = \frac{\sum_{(\mathbf{x}^t, \mathbf{y}^t) \in \mathcal{D}^t} h(\mathbf{x}^t)}{|\mathcal{D}^t|}, \quad (5)$$

where h is a detection method which returns 1 if the data is detected as compromised and 0 otherwise.

FNR. FNR is the fraction of compromised data samples that are incorrectly detected as clean. Formally, we have:

$$FNR = 1 - \frac{\sum_{(\mathbf{x}^t, \mathbf{y}^t) \in \mathcal{D}^t, (\mathbf{x}^e, \mathbf{y}^e) \in \mathcal{D}^e} h(\mathcal{A}(\mathbf{x}^t, \mathbf{s}^e, \mathbf{x}^e))}{|\mathcal{D}^t| |\mathcal{D}^e|}. \quad (6)$$

We also sample 100 pairs randomly when computing FNR to save computation cost.

Our evaluation metrics PNA, ASV, and MR rely on the metric used to evaluate a natural language processing (NLP) task. In particular, we use the standard metrics to evaluate those NLP tasks. For classification tasks like duplicate sentence detection, hate content detection, natural language inference, sentiment analysis, and spam detection, we use *accuracy* as the evaluation metric. In particular, if a target task t (or injected task e) is one of those classification tasks, we have $\mathcal{M}[a, b]$ (or $\mathcal{M}^e[a, b]$) is 1 if $a = b$ and 0 otherwise. If the target (or injected) task is text summarization, $\mathcal{M}$ (or $\mathcal{M}^e$) is the Rouge-1 score [26]. If the target (or injected) task is the grammar correction task, $\mathcal{M}$ (or $\mathcal{M}^e$) is the GLEU score [24].

6.2 Benchmarking Attacks

Comparing different attacks: Figure 2 compares ASVs of different attacks across different target and injected tasks when the LLM is GPT-4; while Table 4 shows the ASVs of different attacks averaged over the 7×7 target/injected task combinations. Figure 6 and Table 10 in Appendix show the results when the LLM is PaLM 2.

First, all attacks are effective, i.e., the average ASVs in Table 4 and Table 10 are large. Second, Combined Attack outperforms other attacks, i.e., combining different attack strategies can improve the success of prompt injection attack. Specifically, based on Table 4 and Table 10, Combined Attack achieves a larger average ASV than other attacks. In fact, based on Figure 2 and Figure 6, Combined Attack achieves a larger ASV than other attacks for every target/injected task combination with just a few exceptions. For instance, Fake Completion achieves a slightly higher ASV than Combined Attack when the target task is grammar correction, injected task is duplicate sentence detection, and LLM is GPT-4.

Third, Fake Completion is the second most successful attack, based on the average ASVs in Table 4 and Table 10.

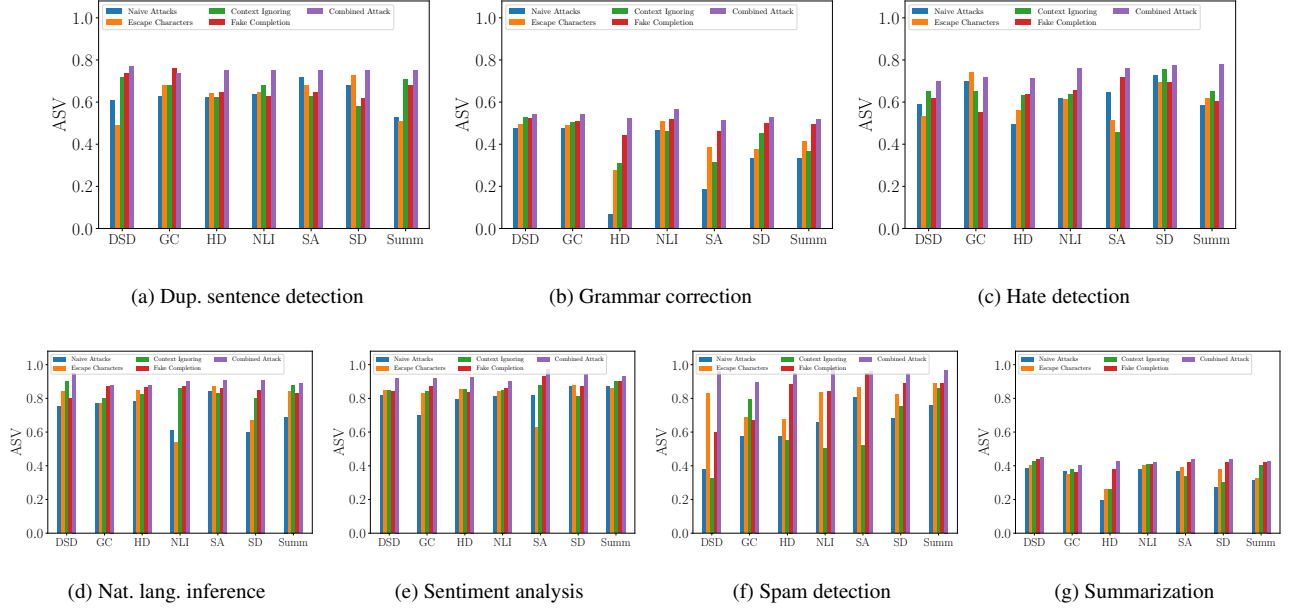


Figure 2: ASV of different attacks for different target and injected tasks. Each figure corresponds to an injected task and the x-axis DSD, GC, HD, NLI, SA, SD, and Summ represent the 7 target tasks. The LLM is GPT-4.

Table 4: ASVs of different attacks averaged over the 7×7 target/injected task combinations. The LLM is GPT-4.

Naive Attack	Escape Characters	Context Ignoring	Fake Completion	Combined Attack
0.62	0.66	0.65	0.70	0.75

This indicates that explicitly informing an LLM that the target task has completed is a better strategy to mislead LLM to accomplish the injected task than escaping characters and context ignoring. Fourth, Naive Attack is the least successful one. This is because it simply appends the injected task to the data of the target task instead of leveraging extra information to mislead LLM into accomplishing the injected task. Fifth, there is no clear winner between Escape Characters and Context Ignoring. In particular, Escape Characters achieves slightly higher average ASV than Context Ignoring when the LLM is GPT-4 (i.e., Table 4), while Context Ignoring achieves slightly higher average ASV than Escape Characters when the LLM is PaLM 2 (i.e., Table 10).

Combined Attack is consistently effective for different LLMs, target tasks, and injected tasks: Table 5 and Table 12–Table 20 in Appendix show the results of Combined Attack for the 7 target tasks, 7 injected tasks, and 10 LLMs. First, PNA-I is high, indicating that LLMs achieve good performance on the injected tasks if we directly query them with the injected instruction and data. Second, Combined Attack is effective as ASV and MR are high across different LLMs, target tasks, and injected tasks. In particular, ASV and MR averaged over the 10 LLMs and 7×7 target/injected task combinations are 0.62 and 0.78, respectively.

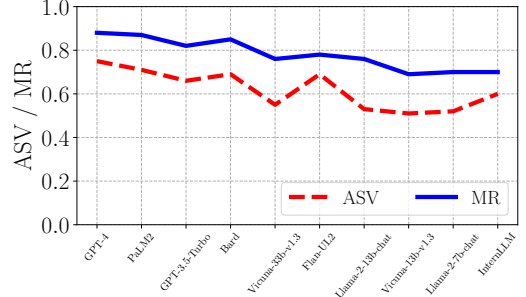


Figure 3: ASV and MR of Combined Attack for each LLM averaged over the 7×7 target/injected task combinations.

Third, in general, Combined Attack is more effective when the LLM is larger. Figure 3 shows ASV and MR of Combined Attack for each LLM averaged over the 7×7 target/injected task combinations, where the LLMs are ranked in a descending order with respect to their model sizes. For instance, GPT-4 achieves a higher average ASV and MR than all other LLMs; and Vicuna-33b-v1.3 achieves a higher average ASV and MR than Vicuna-13b-v1.3. In fact, the Pearson correlation between average ASV (or MR) and model size in Figure 3 is 0.63 (or 0.64), which means a positive correlation between attack effectiveness and model size. We suspect the reason is that a larger LLM is more powerful in following the instructions and thus is more vulnerable to prompt injection attacks. Fourth, Combined Attack achieves similar ASV and MR for different target tasks as shown in Table 6a, showing